

Projenin Kurulumu

Melikşah Camgöz

GitHub: <https://github.com/meliksah-camgoz/Gorev-Yonetim-Projesi-API>

1. Projeyi bilgisayara indirin

Projeyi GitHub üzerinden klonlayabilir veya proje klasörünü bilgisayarınıza kopyalayabilirsiniz.

```
git clone PROJE_URL
```

Ardından proje klasörüne girin:

```
cd task-management-api
```

2. Gerekli paketleri yükleyin

Terminalde:

```
npm install
```

çalıştırın.

Gerekli temel paketler:

- express
- mysql2
- dotenv

Geliştirme sırasında kullanılan paket:

- nodemon

Veritabanı Kurulumu

MySQL üzerinde aşağıdaki veritabanını oluşturun:

```
CREATE DATABASE task_management;
```

```
USE task_management;
```

Employees Tablosu

```
CREATE TABLE employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    role VARCHAR(100),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tasks Tablosu

```
CREATE TABLE tasks (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    description TEXT,  
    assigned_to INT,  
    status ENUM('pending', 'in_progress', 'completed') DEFAULT  
'pending',  
    priority ENUM('low', 'medium', 'high') DEFAULT 'medium',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (assigned_to)  
        REFERENCES employees(id)  
        ON DELETE SET NULL  
);
```

Örnek Çalışan Verileri

Sistemi test etmek için:

```
INSERT INTO employees (name, email, role)
VALUES
('Ahmet Yılmaz', 'ahmet@example.com', 'Backend Developer'),
('Mehmet Demir', 'mehmet@example.com', 'Frontend Developer'),
('Ayşe Kaya', 'ayse@example.com', 'Project Manager');
```

Ortam Değişkenleri

Proje ana dizininde .env dosyası oluşturun:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=MYSQL_SIFRENIZ
DB_NAME=task_management
PORT=3000
```

.env dosyası GitHub'a yüklenmemelidir.

Projeyi Çalıştırma

Geliştirme modunda çalıştırmak için:

```
npm run dev
```

Normal şekilde çalıştırmak için:

```
npm start
```

Sunucu varsayılan olarak:

<http://localhost:3000>

adresinde çalışır.

API Endpointleri

Görevler

Method	Endpoint	Açıklama
GET	/api/tasks	Tüm görevleri getirir
GET	/api/tasks/:id	ID'ye göre görev getirir
POST	/api/tasks	Yeni görev oluşturur
PUT	/api/tasks/:id	Görevi günceller
DELETE	/api/tasks/:id	Görevi siler

Görev Oluşturma

Request

POST /api/tasks

Body

```
{
  "title": "Login API geliştirme",
  "description": "Kullanıcı giriş API'si hazırlanacak.",
  "assigned_to": 1,
  "status": "pending",
  "priority": "high"
}
```

Başarılı Response

```
{
  "message": "Görev başarıyla oluşturuldu.",
  "taskId": 1
}
```

Görev Listeleme

GET /api/tasks

Görev Detayı

GET /api/tasks/1

Görev Güncelleme

PUT /api/tasks/1

Body

```
{
  "title": "Login API tamamlandı",
  "description": "Login API geliştirme işlemi tamamlandı.",
  "assigned_to": 1,
  "status": "completed",
  "priority": "high"
}
```

Görev Silme

DELETE /api/tasks/1

Başarılı response:

```
{
  "message": "Görev başarıyla silindi."
}
```

Görev Filtreleme

Görevler çeşitli kriterlere göre filtrelenebilir.

Duruma göre

GET /api/tasks?status=pending

Önceliğe göre

GET /api/tasks?priority=high

Çalışana göre

GET /api/tasks?assigned_to=1

Birden fazla filtre

GET /api/tasks?status=pending&priority=high

Görev Durumları

Sistemde üç farklı görev durumu bulunmaktadır:

Değer	Açıklama
pending	Bekliyor
in_progress	Devam ediyor
completed	Tamamlandı

Görev Öncelikleri

Değer	Açıklama
low	Düşük
medium	Orta
high	Yüksek

Middleware

Projede tüm API isteklerini takip etmek amacıyla Logger Middleware kullanılmaktadır.

Logger;

- HTTP Method
- Endpoint
- Zaman damgası

bilgilerini konsola yazdırmaktadır.

Örnek:

```
[2026-09-23T08:06:00.000Z] GET /api/tasks  
[2026-09-23T08:07:12.000Z] POST /api/tasks  
[2026-09-23T08:08:35.000Z] PUT /api/tasks/1
```

Ayrıca görev oluşturma ve güncelleme işlemlerinde validation middleware kullanılarak gelen verilerin kontrolü yapılmaktadır.

Postman ile Test

API endpointleri Postman kullanılarak test edilebilir.

Test edilmesi gereken temel işlemler:

1. GET - Tüm görevleri getir
2. GET - ID ile görev getir
3. POST - Yeni görev oluştur
4. PUT - Görev güncelle
5. DELETE - Görev sil
6. GET - Duruma göre filtrele
7. GET - Önceliğe göre filtrele
8. GET - Çalışana göre filtrele

HTTP Durum Kodları

Projede temel HTTP durum kodları kullanılmaktadır:

Kod	Açıklama
200	Başarılı
201	Başarıyla oluşturuldu
400	Hatalı istek
404	Kayıt bulunamadı
500	Sunucu hatası

Klasör Yapısı

```
task-management-api/  
├── src/  
│   ├── config/  
│   │   └── db.js  
│   ├── controllers/  
│   │   └── taskController.js  
│   ├── middleware/  
│   │   └── logger.js
```

```
├── taskValidation.js
├── routes/
│   └── taskRoutes.js
├── app.js
├── .env
├── .gitignore
├── package.json
├── package-lock.json
└── README.md
```